

**BTS Services Informatiques aux Organisations (SIO)**  
***Session 2025***

**Nom du candidat :** Da Silva Peixoto

**Prénom du candidat :** Gabriel

**Parcours :** BTS SIO option SLAM

***Fiche de réalisation professionnelle N° 5***

**Déploiement d'une Application en JavaFX**

**Période de réalisation :** 03/03/26 au 04/02/26

**Type de réalisation :** En Stage, 2ème année.

**Mode de réalisation :** Individuelle

## **Sommaire**

|                                           |          |
|-------------------------------------------|----------|
| <b>Contexte organisationnel</b>           | <b>3</b> |
| <b>Problématique</b>                      | <b>3</b> |
| <b>Choix de la solution mise en œuvre</b> | <b>3</b> |
| <b>Résolution</b>                         | <b>3</b> |
| <b>Conclusions</b>                        | <b>4</b> |
| <b>Compétences</b>                        | <b>4</b> |

## CONTEXTE ORGANISATIONNEL

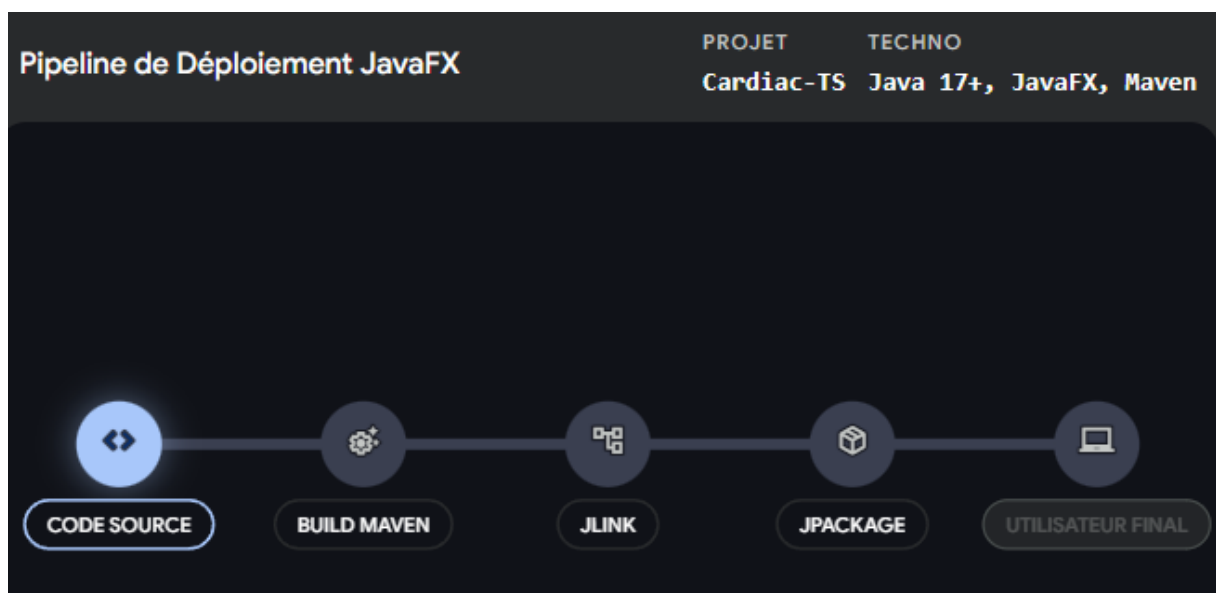
Dans le cadre de mon immersion professionnelle au sein du Laboratoire TIMC (CNRS), j'ai été missionné pour concevoir et développer de bout en bout Cardiac-TS, une application de bureau complexe destinée à la recherche médicale. Ce logiciel, doté d'une interface graphique développée en JavaFX et respectant strictement le patron de conception MVC (avec une couche Service en plus), permet aux chercheurs de traiter, visualiser et gérer des données cliniques et des fichiers d'imagerie médicale particulièrement lourds. L'environnement de ce laboratoire de recherche est très spécifique : les utilisateurs finaux (médecins, biologistes, data scientists) sont des experts dans leur domaine, mais ils ne disposent pas nécessairement de compétences techniques approfondies en ingénierie logicielle. De plus, le parc informatique du laboratoire est hétérogène et soumis à des règles de sécurité strictes, interdisant souvent l'installation manuelle de multiples dépendances ou la modification des variables d'environnement par les utilisateurs eux-mêmes.

## PROBLÉMATIQUE

La distribution d'un client lourd développé en Java a historiquement toujours posé des problèmes majeurs de portabilité et d'expérience utilisateur. Dans une approche classique, le chercheur aurait dû télécharger et installer la bonne version du Java Runtime Environment (JRE), configurer ses variables d'environnement, puis lancer l'application via une ligne de commande ou un fichier exécutable souvent instable. Cette méthode archaïque génère inévitablement des conflits de versions, des erreurs de librairies manquantes, ou des blocages liés aux droits d'administration. L'enjeu critique pour moi n'était donc pas seulement de coder l'application, mais de garantir que son déploiement soit totalement transparent et infaillible. Il fallait transformer un projet de développement complexe, avec ses dizaines de dépendances (pilotes PostgreSQL, clients MinIO, bibliothèques graphiques), en un livrable unique, autonome, et déployable en un seul clic, sans exiger la moindre intervention technique de la part du corps médical.

## CHOIX DE LA SOLUTION MISE EN ŒUVRE

Pour rompre définitivement avec ces contraintes de déploiement, j'ai pris l'initiative d'exploiter les outils de packaging natifs introduits dans les versions récentes du Java Development Kit (JDK), en abandonnant la création classique de fichiers JAR exécutables. Mon choix s'est porté sur l'association stratégique de deux outils : `jlink` et `jpackage`. L'outil `jlink` offre la possibilité d'analyser le code source pour créer une machine virtuelle Java personnalisée et miniaturisée, ne contenant que les modules strictement nécessaires au fonctionnement de Cardiac-TS, ce qui réduit drastiquement le poids du livrable. Dans un second temps, `jpackage` permet d'encapsuler cette image d'exécution optimisée, ainsi que tout le code de l'application, au sein d'un véritable installateur natif propre au système d'exploitation cible (comme un fichier `.msi` ou `.exe` pour Windows). Cette architecture garantit une isolation totale : l'application embarque son propre moteur d'exécution et devient totalement agnostique vis-à-vis de la configuration initiale du poste client.



## RÉSOLUTION

La mise en œuvre de cette stratégie de déploiement :

J'ai configuré de manière exhaustive le fichier `module-info.java` pour déclarer explicitement quelles bibliothèques externes étaient requises et quelles parties de mon code devaient être exposées. En parallèle, j'ai optimisé mon fichier de configuration Maven (`pom.xml`) pour qu'il gère la résolution des dépendances transitives sans conflit. Une fois cette architecture modulaire stabilisée, j'ai pu exécuter l'outil `jlink`. Cette commande a généré un dossier contenant une image d'exécution allégée, débarrassée de tous les outils de développement et bibliothèques superflus du JDK standard, garantissant ainsi des temps de démarrage de l'application beaucoup plus rapides.

Dans un second temps, je me suis concentré sur la création de l'installateur final destiné aux chercheurs. En utilisant la ligne de commande `jpackage`, j'ai pointé vers l'image générée par `jlink` et j'y ai injecté les paramètres spécifiques à notre environnement. J'ai pu définir des métadonnées professionnelles, intégrer le logo du laboratoire en guise d'icône d'application, et configurer la création automatique de raccourcis sur le bureau de l'utilisateur. L'objectif sera de se connecter aux conteneurs Docker (PostgreSQL et MinIO) hébergés sur le réseau du TIMC, et que l'interface graphique répondait sans latence.

## CONCLUSIONS

L'objectif de l'application de CardiacTS est la portabilité. L'application est passée du statut de prototype technique complexe à celui d'un produit logiciel professionnel "Plug-and-Play". Le temps d'installation pour un nouveau collaborateur ou un chercheur a été réduit à quelques secondes, éliminant totalement les demandes d'assistance technique liées aux erreurs de configuration Java. Cette démarche m'a prouvé que le métier de développeur ne s'arrête pas à la production d'un code fonctionnel, mais englobe toute la chaîne de valeur jusqu'à la mise à disposition fluide et sécurisée du service pour l'utilisateur final. C'est une compétence d'ingénierie logicielle que je compte réinvestir systématiquement dans mes futurs projets.

## **COMPÉTENCES**

**B1.5.1 Déployer un service : Création d'une image d'exécution modulaire (jlink) et génération d'un installeur natif autonome (jpackage) pour distribuer le client lourd Cardiac-TS.**

**B1.5.2 Réaliser les tests d'intégration et d'acceptation d'un service : Déploiement et validation du fichier d'installation sur des environnements virtuels vierges pour garantir l'absence de dépendances externes.**

**B1.5.3 Accompagner les utilisateurs dans la mise en place d'un service : Rédaction d'une documentation technique simplifiée et accompagnement des chercheurs lors de la prise en main de l'exécutable.**